

Дизайн и анализ на алгоритми упражнение 2

Станислав Игов (Стъката)

2025/2026

Размер на входа. Псевдокод. FindMin алгоритъм. Инвариант. Алгоритъм на Евклид.

1 Преговор от миналия път

UCM (Unit Cost Measure) – При този модел всяка елементарна операция струва 1.

LCM (Logarithmic Cost Measure) – Тук цената на операцията зависи от размера на числата.

Миналият път дефинирахме класовете

$$O(g) = \{ f \mid f \preceq g \}.$$

$$\Omega(g) = \{ f \mid g \preceq f \}.$$

$$\Theta(g) = O(g) \cap \Omega(g).$$

2 Размер на входа

Какво наричаме числа и масиви?

Числата могат да са от множествата $\mathbb{N}$, $\mathbb{Z}$ и $\mathbb{Q}$. При $\mathbb{R}$ възниква въпросът за представянето чрез апроксимация с крайна точност (тази тема може да бъде засегната по-късно).

Масивите са крайни редици от числа или от други масиви.

Нека се разберем, че работим с **UCM**, т.е. за нас размерът на всяко число е 1.

Ще дефинираме размера рекурсивно.

Нека

$$S : \mathbb{N} \cup \mathcal{M} \rightarrow \mathbb{N}^+.$$

където $\mathcal{M}$ е множеството от всички масиви.

Размер на число

$$S_N(n) = 1 \quad \text{за всяко } n \in \mathbb{N}.$$

Размер на масив

Нека

$$M = \{M_i\}_{i=1}^m, \quad \text{където } m \in \mathbb{N}^+.$$

Тогава размерът на масива се дефинира чрез

$$S_M(M) = \sum_{i=1}^m S(M_i).$$

Обща рекурсивна дефиниция

$$S(a) = \begin{cases} S_N(a), & \text{ако } a \in \mathbb{N}, \\ S_M(a), & \text{ако } a \in \mathcal{M}. \end{cases}$$

Тоест:

ако е число $\rightarrow$ размерът е 1;

ако е масив $\rightarrow$ размерът е сумата от размерите на неговите елементи.

Размерът на редица от числа и масиви

$$\{E_i\}_{i=1}^n$$

е сумата от размерите на елементите ѝ.

3 Псевдокод

3.1 Какво е псевдокод?

Опростен език за описание на алгоритми.

Няма строг синтаксис – правилата ги определяме ние.

Близък до Pascal.

3.2 Основни правила

3.2.1 Именуване на променливи

Имена на масиви $\rightarrow$ главна буква

Имена на променливи $\rightarrow$ малка буква

3.2.2 Присвояване и сравнение

Присвояване: $\leftarrow$

Сравнение: $=$

3.2.3 Масиви

Обхват: $A[1..n]$

Двумерен масив: $A[1..k, 1..n]$ или $A[1..k][1..n]$

Индексирането започва от 1 (освен ако не е дефинирано друго).

Допускат се:

$A[m, n]$

$A[m][n]$

3.2.4 Условни конструкции

```
if условие then
    инструкции
else
    инструкции
endif
```

`endif` отбелязва края на условната конструкция.

Използват се табулации за яснота.

3.2.5 while цикъл

```
while условие do
    инструкции
endwhile
```

Проверява условието преди всяка итерация.

`endwhile` отбелязва края.

3.2.6 do-while (цикъл с постусловие)

```
do
    инструкции
while условие
```

Ново изпълнение има само ако условието е True.

Условието се проверява след изпълнение на тялото.

3.2.7 for цикъл

```
for i ← 1 to n do
    инструкции
endfor
```

Цикълът се изпълнява от 1 до n включително.

`endfor` отбелязва края на цикъла.

3.2.8 Булеви изрази

Конюнкция $\rightarrow$ **and**

Дизюнкция $\rightarrow$ **or**

Оценяване отляво надясно (short-circuit):

При **and**: ако първото е False $\rightarrow$ останалите не се проверяват.

При **or**: ако първото е True $\rightarrow$ останалите не се проверяват.

3.2.9 Блокове код

Използваме означители:

endif

endwhile

endfor

Използваме и табулиране за по-добра четимост.

4 Доказателство на итеративен алгоритъм

4.1 Коректност на алгоритъм

Дефиниция. Казваме, че алгоритъм е коректен, ако за всеки негов вход той:

- завършва (терминира)
- връща правилен резултат.

4.2 Инвариант

Дефиниция. Инвариант е свойство на даден обект, което остава непроменено след дадена операция или трансформация на този обект.

4.3 Верификация на итеративни алгоритми

Верификацията на итеративни алгоритми се извършва чрез **инвариант на цикъла**.

Доказателствата за коректност с инвариант по същество са доказателства по индукция по броя на “завъртанията” на цикъла.

Инвариантът е едноместен предикат, свързан с цикъла. Променливата на предиката е броят на достиганията на дадена инструкция на алгоритъма. Обикновено това е инструкцията, която съдържа условието за край на цикъла.

Нека условието за край на цикъла е на ред l . Предикатът трябва да бъде:

- верен първия път, когато изпълнението на алгоритъма достигне ред l ;
- запазващ своята вярност при всяко следващо изпълнение (оттук идва името “инвариант”);
- при напускане на цикъла неговата вярност да влече директно твърдението, което искаме да докажем за алгоритъма.

4.4 Идея за формулиране на инвариант

Инвариантът обикновено има вида:

При k -тото достигане на ред l на алгоритъма е изпълнено някакво твърдение, формулирано чрез k .

4.5 Стъпки за доказване на инварианта

- **База.** Показваме, че $P(1)$ е истина при първото достигане на въпросния ред.
- **Поддръжка.** Допускайки, че $P(k)$ е в сила за някое достигане, което не е последното, показваме, че $P(k + 1)$ е в сила при следващото достигане.
- **Терминация.** Разглеждаме момента, в който изпълнението достига въпросния ред за последен път. Нека k' е броят на всички достигания до този ред, включително последното. Замествайки k с k' в инварианта, получаваме точно твърдението за алгоритъма, което искаме да докажем.

5 Алгоритъм FindMin

Алгоритъмът FindMin при подаден масив от цели числа $A[1..n]$ връща индекса на най-малкия елемент в масива.

```
FindMin(A[1..n]): масив от цели числа)
@1 m = 1
@2 for j <- 2 to n do
@3     if A[m] > A[j] then
@4         m <- j
@5     endif
@6 endfor
@7 return m
```

Нека m_i е стойността на m непосредствено преди изпълнението на i -тата итерация на цикъла. Нека $i = j - 1$, тъй като работим с j , започвайщо от 2. Тогава $j = i + 1$.

Дефинираме инварианта $I(i)$ чрез:

$$I(i) \iff \begin{cases} A[m_i] = \min A[1..i] \\ 1 \leq m_i \leq j \end{cases}$$

Доказателство.

База. Нека $i = 1$. Тогава $m_1 = 1$. Имаме

$$A[m_1] = A[1] = \min A[1..1].$$

С това първото условие е изпълнено. Освен това

$$1 \leq m_1 = 1 < 2 = j,$$

следователно второто условие също е изпълнено.

Поддръжка. Нека инвариантът е верен за $I(i)$. Ще докажем, че е верен за $I(i + 1)$.

Разглеждаме двата възможни случая според условието на ред @3.

Първи случай: условието е вярно, тоест $A[m_i] > A[j] = A[i + 1]$. Тогава $m_{i+1} = i + 1$. Понеже по индукционно предположение $A[m_i] = \min A[1..i]$ и $A[i + 1]$ е по-малко от този минимум, получаваме

$$A[m_{i+1}] = \min A[1..i + 1].$$

Освен това $1 \leq m_{i+1} = i + 1 \leq j$, следователно и второто условие е изпълнено.

Втори случай: условието е невярно, тоест $A[m_i] \leq A[j] = A[i + 1]$. Тогава

$m_{i+1} = m_i$. От индукционното предположение следва, че

$$A[m_{i+1}] = A[m_i] = \min A[1..i] = \min A[1..i + 1],$$

и неравенството за индекса също остава в сила. Следователно инвариантът се запазва.

Терминация: Нека цикълът приключи. Тогава $j = n + 1$ и понеже $i = j - 1$, получаваме $i = n$. От инварианта следва

$$A[m_n] = \min A[1..n],$$

и $1 \leq m_n \leq n$. Следователно алгоритъмът връща индекса на минималния елемент. С това доказателството за коректност приключва.

Анализ на времевата сложност.

Ред @1 се изпълнява 1 път.

Ред @2 се проверява n пъти.

Ред @3 се изпълнява $n - 1$ пъти.

Ред @4 в най-лошия случай се изпълнява $n - 1$ пъти.

Ред @7 се изпълнява 1 път.

Общият брой е пропорционален на $3n$, следователно

$$\text{Time}(\text{FindMin}) \in O(n).$$

От друга страна, не можем да намерим минимален елемент, без да разгледаме всички елементи на масива. Следователно

$$\text{Time}(\text{FindMin}) \in \Omega(n).$$

Оттук следва

$$\text{Time}(\text{FindMin}) \in \Theta(n).$$

6 Алгоритъм на Евклид

Пример: 2260, 816

$$\begin{aligned}2260 &= 2 \cdot 816 + 628 \\816 &= 1 \cdot 628 + 188 \\628 &= 3 \cdot 188 + 64 \\188 &= 2 \cdot 64 + 60 \\64 &= 1 \cdot 60 + 4 \\60 &= 15 \cdot 4 + 0\end{aligned}$$

Следователно $\text{GCD}(2260, 816) = 4$.

Вход: a, b естествени числа. Изход: $\text{GCD}(a, b)$.

```
EuclidGCD(a,b : естествени числа)
@1 while a > 0 and b > 0 do
@2     if a > b then
@3         a <- a mod b
@4     else
@5         b <- b mod a
@6     endif
@7 endwhile
@8 return max(a,b)
```

Ще докажем, че при вход естествени числа a, b алгоритъмът връща техния най-голям общ делител.

Нека a_i и b_i са стойностите на a и b при i -тата итерация на цикъла на ред @1 и нека

$$d = \text{GCD}(a, b).$$

Дефинираме инварианта

$$I(i) \iff \text{GCD}(a_i, b_i) = d.$$

Ще докажем с индукция по $i \geq 1$, че $I(i)$ е вярно.

База $i = 1$. Тогава $a_1 = a$ и $b_1 = b$, следователно

$$\text{GCD}(a_1, b_1) = \text{GCD}(a, b) = d.$$

Поддръжка. Нека $I(i)$ е вярно. Разглеждаме изпълнението на следващата итерация.

Първи случай: $a_i > b_i$. Тогава

$$a_i = qb_i + r, \quad r = a_i - qb_i,$$

и алгоритъмът присвоява

$$a_{i+1} = r = a_i - qb_i, \quad b_{i+1} = b_i.$$

Тогава

$$\text{GCD}(a_{i+1}, b_{i+1}) = \text{GCD}(a_i - qb_i, b_i) = \text{GCD}(a_i, b_i) = d.$$

Тук използваме свойството: ако $d \mid x$ и $d \mid y$, то $d \mid (x - ky)$

Втори случай: $b_i > a_i$ е напълно аналогичен.

Следователно инвариантът се запазва.

Терминация. Нека цикълът спре при n -тата итерация. Тогава $a_n = 0$ или $b_n = 0$.

Знаем, че

$$\text{GCD}(0, k) = k \quad \text{за всяко } k \geq 1.$$

На ред @8 алгоритъмът връща $\max(a_n, b_n)$, което е точно $\text{GCD}(a_n, b_n) = d$.

Следователно алгоритъмът е коректен.

Остава да покажем, че алгоритъмът завършва.

Разглеждаме сумата $a_i + b_i$. Имаме

$$a_{i+1} + b_{i+1} < a_i + b_i,$$

тъй като остатъкът винаги е по-малък от делителя. Следователно редицата $a_i + b_i$ е строго намаляваща редица от естествени числа, което означава, че алгоритъмът задължително ще завърши.

Следователно **EuclidGCD** завършва коректно за всеки вход.

Времева сложност.

Наблюдаваме примери:

$a = 0, b = 5 - 1$ стъпка.

$a = 128, b = 16 - 2$ стъпки.

$a = 34, b = 21 - 8$ стъпки.

Последният пример използва две последователни числа на Фибоначи.

База: $N = 1$

Ако алгоритъмът прави точно една итерация, това означава, че след първото деление остатъкът става 0, т.е. по-малкото число дели по-голямото. Тъй като $a > b$, това може да стане само ако:

$$b \mid a.$$

Най-малката възможна такава двойка положителни числа е:

$$a \geq 2, \quad b \geq 1.$$

Но:

$$f_1 = 1, \quad f_2 = 1, \quad f_3 = 2.$$

Следователно:

$$a \geq f_3 = f_{1+2}, \quad b \geq f_2 = f_{1+1}.$$

Базата е вярна.

Индукционна хипотеза

Нека твърдението е вярно за всички $N' < N$. Тоест ако алгоритъмът спре за N' стъпки, тогава:

$$a \geq f_{N'+2}, \quad b \geq f_{N'+1}.$$

Индукционна стъпка

Нека алгоритъмът върху вход (a, b) прави точно N стъпки.

След първата итерация получаваме нова двойка:

$$(a_1, b_1) = (b, a \bmod b).$$

Тази нова двойка ще доведе до точно $N - 1$ стъпки. От индукционната хипотеза за нея следва:

$$a_1 \geq f_{N+1}, \quad b_1 \geq f_N.$$

Но:

$$a_1 = b, \quad b_1 = a - qb \quad \text{за някакво } q \geq 1.$$

Понеже остатъкът е по-малък от делителя, имаме:

$$a = qb + b_1 \geq b + b_1.$$

Следователно:

$$a \geq f_{N+1} + f_N.$$

По дефиниция на числата на Фибоначи:

$$f_{N+2} = f_{N+1} + f_N,$$

$$f_{n+2} = f_{n+1} + f_n.$$

Затова:

$$a \geq f_{N+2}.$$

Освен това:

$$b = a_1 \geq f_{N+1}.$$

Индукционната стъпка е доказана.

Заклучение

Ако алгоритъмът прави N стъпки, то входните числа трябва да са поне толкова големи, колкото съответните числа на Фибоначи:

$$a \geq f_{N+2}, \quad b \geq f_{N+1}.$$

Следствие: оценка на сложността

Числата на Фибоначи растат експоненциално:

$$f_n \approx \varphi^n, \quad \text{където } \varphi = \frac{1 + \sqrt{5}}{2}.$$

Следователно:

$$b \geq f_{N+1} \approx \varphi^{N+1}.$$

Логаритмуваме:

$$N = O(\log b).$$

Тъй като $b \leq \min(a, b)$, получаваме стандартната оценка:

EuclidGCD работи за $O(\log \min(a, b))$.